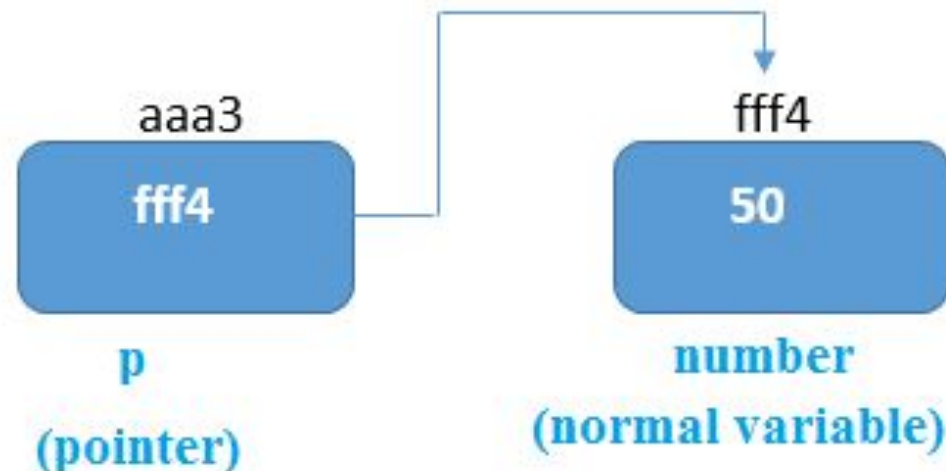


Pointer

Prepared by:
Antu Chowdhury
Lecturer, Dept. of CSE

Introduction

- **Pointers are one of the core components of the C programming language. A pointer can be used to store the memory address of other variables, functions, or even other pointers. The use of pointers allows low-level memory access, dynamic memory allocation, and many other functionality in C.**
- **An example of using pointers to print the address and value is given below.**



Syntax of C Pointers

The syntax of pointers is similar to the variable declaration in C, but we use the (*) dereferencing operator in the pointer declaration.

datatype * ptr;

- Where,

ptr is the name of the pointer.

datatype is the type of data it is pointing to.

- The above syntax is used to define a pointer to a variable.
We can also define pointers to functions, structures, etc.

How to Use Pointers?

The use of pointers in C can be divided into three steps:

- **Pointer Declaration**
- **Pointer Initialization**
- **Pointer Dereferencing**

1. Pointer Declaration

- In pointer declaration, we only declare the pointer but do not initialize it. To declare a pointer, we use the (*) dereference operator before its name.
- Example

*int *ptr;*

- The pointer declared here will point to some random memory address as it is not initialized. Such pointers are called wild pointers.

2. Pointer Initialization

- **Pointer initialization is the process where we assign some initial value to the pointer variable. We generally use the (&) address of operator to get the memory address of a variable and then store it in the pointer variable.**
- **Example**

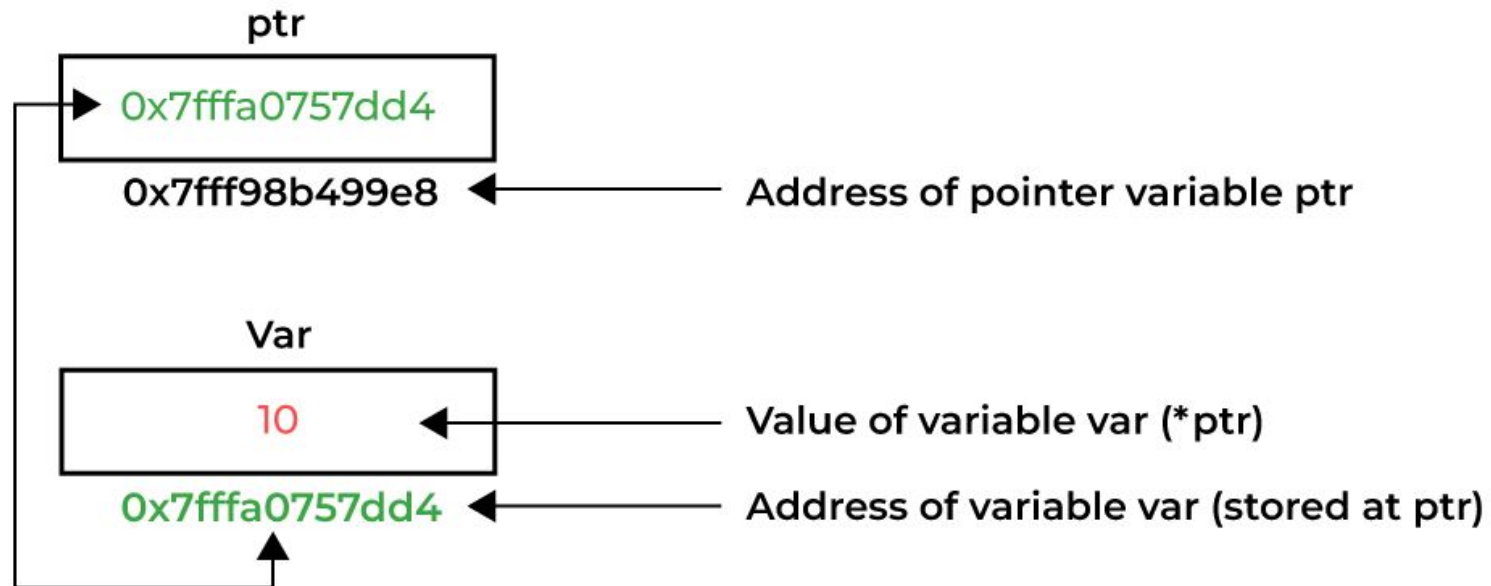
int var = 10;

int * ptr;

ptr = &var;

3. Pointer Dereferencing

Dereferencing a pointer is the process of accessing the value stored in the memory address specified in the pointer. We use the same (*) dereferencing operator that we used in the pointer declaration.



C Pointer Example

```
// C program to illustrate Pointers
#include <stdio.h>
int main()
{
    int var = 10;
    int* ptr;
    ptr = &var;
    printf("Value at ptr = %p \n", ptr);
    printf("Value at var = %d \n", var);
    printf("Value at *ptr = %d \n", *ptr);
    return 0;
}
```

Value at ptr = 0x7fff1038675c

Value at var = 10

Value at *ptr = 10

Null pointer in c

- A null pointer is a pointer which points nothing.
- Some uses of the null pointer are:
 - a) To initialize a pointer variable when that pointer variable isn't assigned any valid memory address yet.
 - b) To check for null pointer before accessing any pointer variable. So that, we can perform **error handling** in pointer related code e.g. dereference pointer variable only if it's not NULL.

Example:

```
#include <stdio.h>

int main() {
    int *p= NULL;//initialize the pointer as null.
    printf("The value of pointer is %d",p);
    return 0;
}
```

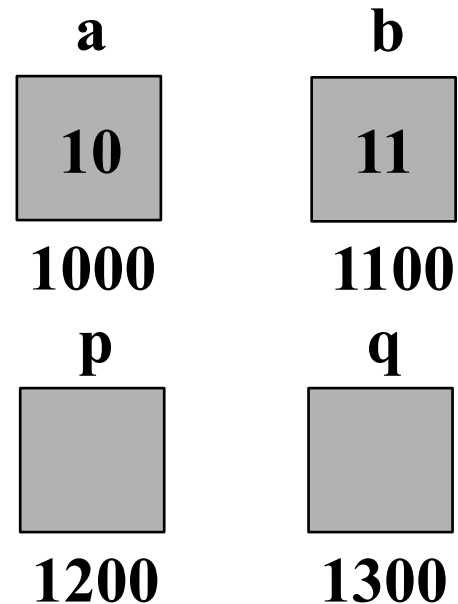
The value of pointer is 0.

Example: Null pointer in c

- **#include <stdio.h>**
- **// Function that tries to find a number in an array.**
- **// Returns a pointer to the element, or NULL if not found.**
- **int* findNumber(int arr[], int size, int key) {**
- **for (int i = 0; i < size; i++) {**
- **if (arr[i] == key) {**
- **return &arr[i]; // return address of found element**
- **}**
- **}**
- **return NULL; // not found**
- **}**
- **int main() {**
- **int numbers[] = {10, 20, 30, 40, 50};**
- **int key = 25;**
- **int *result = findNumber(numbers, 5, key);**
- **if (result != NULL) {**
- **printf("Found: %d\n", *result); // safe to dereference**
- **} else {**
- **printf("Number %d not found in array.\n", key);**
- **}**
- **return 0;**
- **}**

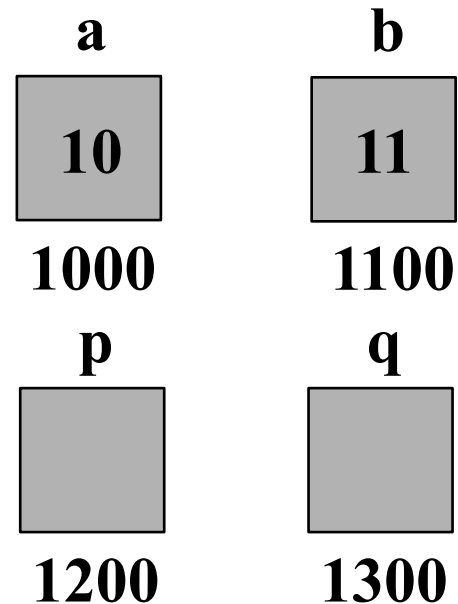
Pointer Assignment

- `#include<stdio.h>`
- `int main()`
- `{`
- `int a=10, b=11;`
- `int *p, *q;`
- `p=&a;`
- `q=p;`
- `printf("%d %d %d ",a, *p, *q);`
- `}`



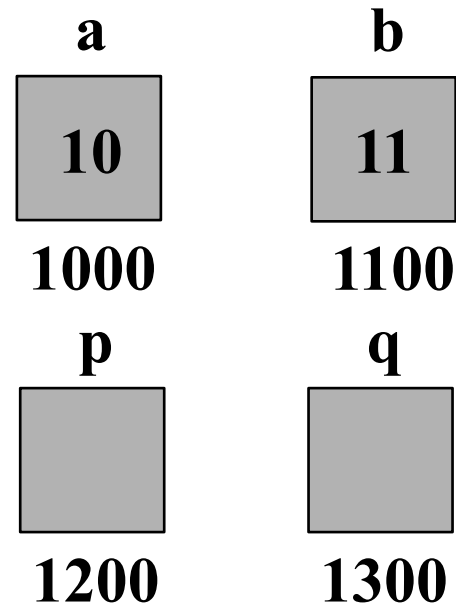
Pointer Assignment (contd.)

- `#include<stdio.h>`
- `int main()`
- `{`
- `int a=10, b=11;`
- `int *p, *q;`
- `p=&a;`
- `*q=*p;`
- `printf("%d %d %d ",a, *p, *q);`
- `}`



Pointer Assignment (contd.)

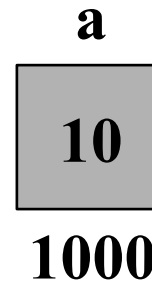
- `#include<stdio.h>`
- `int main()`
- `{`
- `int a=10, b=11;`
- `int *p, *q;`
- `p=&a;`
- `q=&b;`
- `*q=*p;`
- `printf("%d %d %d %d ",a, b, *p, *q);`
- `}`



Pointer arithmetic (Addition)

✗ When Pointer Arithmetic Does NOT Work

- `int a=10;`
- `int *p=&a;`
- `p=p+1;` // ✗ Moving pointer past 'a'
- `printf("%d",*p);` // ✗ Dereferencing invalid memory



Pointer arithmetic (Addition)

-  **Correct Usage (with array):**
- **If you want to use pointer arithmetic safely, use an array:**
- `#include<stdio.h>`
- `int main()`
- `{`
- `int arr[] = {10, 20, 30};`
- `int *p = &arr[0];`
- `p = p + 1;`
- `printf("%d", *p);`
- `return 0;`
- `}`



Key Concept:

`p + n` moves the pointer forward by `n * sizeof(type)` bytes.

Example: Pointer arithmetic (Addition)

- `#include <stdio.h>`
- `int main() {`
- `int arr[] = {10, 20, 30, 40};`
- `int *p = &arr[0];`
- `printf("%d\n", *p); // 10`
- `printf("%d\n", *(p + 1)); // 20`
- `printf("%d\n", *(p + 2)); // 30`
- `printf("%d\n", *p+3); // ?`
- `return 0;`
- `}`

Conclusion: Pointer arithmetic (Addition)

Expression	Meaning	Points to	Value
p	&arr[0]	0th element	10
p + 1	&arr[1]	1st element	20
p + 2	&arr[2]	2nd element	30
*(p + 2)	Value at arr[2]	—	30

Example: Pointer arithmetic (Subtraction)

- **#include <stdio.h>**
- **int main() {**
- **int arr[] = {10, 20, 30, 40};**
- **int *p = &arr[2];**
- **printf("%d\n", *(p - 1)); // 20**
- **printf("%d\n", *(p - 2)); // 10**
- **return 0;**
- **}**

Pointer arithmetic (Pointer Increment & Decrement)

Basic Concept

- If `p` is a pointer to type `T` (e.g., `int`), then:
 - `p++` moves the pointer to the next memory location of type `T`
 - `p--` moves it to the previous location of type `T`
- Not just by 1 byte — it moves by `sizeof(T)` bytes
- `#include <stdio.h>`
- `int main() {`
- `int arr[] = {10, 20, 30, 40};`
- `int *p = &arr[0];`
- `printf("%d\n", *p); // 10`
- `p++; // move to arr[1]`
- `printf("%d\n", *p); // 20`
- `printf("%d\n", *p++); // 20`
- `p--; // move back to arr[1]`
- `printf("%d\n", *p); // 20`
- `printf("%d\n", *++p); // 30`
- `printf("%d\n", *--p); // 20`
- `return 0;`
- `}`

Example: Pointer arithmetic (Pointer Increment & Decrement)

```
int arr[] = {10, 20, 30, 40};  
    int *p = &arr[0];  
printf("%d %d",*p++, *p++);//20//10
```

```
int arr[] = {10, 20, 30, 40};  
    int *p = &arr[0];  
printf("%d %d",*++p, *++p);//30//20
```

```
int arr[] = {10, 20, 30, 40};  
    int *p = &arr[0];  
printf("%d %d",*p++, *++p);//20//20
```

```
int arr[] = {10, 20, 30, 40};  
    int *p = &arr[0];  
printf("%d %d",*++p, *p++);//30//10
```

C – Pointer to Pointer (Double Pointer)

- The pointer to a pointer in C is used when we want to store the address of another pointer. The first pointer is used to store the address of the variable. And the second pointer is used to store the address of the first pointer. That is why they are also known as *double-pointers*.

Representation of Double pointer

Suppose,
`int a = 80;`
`int *p = &a;`
`int **q=&p;`

****q explanation:**

***q will give the value at location 2000, which is 1000.**

****q will give the value at location 1000, which is 80**

a = 80		p = 1000		q = 2000
1000		2000		3000

Hey, a is a variable stored at 1000 location, p is a pointer which contains the address of a variable and q is a double pointer which contains the address of p pointer.

Value of **q = 80
Value of *p = 80
Value of a = 80
Value of p = 1000
Value of q = 2000
Value of &a = 1000

Code Example

// C program to demonstrate pointer to pointer

#include <stdio.h>

int main()

{

int var = 789;

// pointer for var

int* ptr2;

// double pointer for ptr2

int ptr1;**

// storing address of var in ptr2

ptr2 = &var;

// Storing address of ptr2 in ptr1

ptr1 = &ptr2;

printf("Value of var = %d\n", var);

printf("Value of var using single pointer = %d\n", *ptr2);

printf("Value of var using double pointer = %d\n", **ptr1);

return 0;

}

Value of var = 789

Value of var using single pointer = 789

Value of var using double pointer = 789